Why a transaction with a start but no commit must be undone

A transaction is atomic: either all of its updates take effect or none do. A <T start> record without a matching <T commit> means the system failed before the transaction finished, so the database never reached a point where its work was promised to be permanent.

Because buffer pages may be flushed to disk at any time, some of the transaction's writes may already be on disk even though it never committed. Leaving them would expose partial, inconsistent results (for example, a debit without its matching credit) and would violate atomicity and consistency.

Recovery therefore treats such a transaction as incomplete and scans the log backward, restoring each variable to its old value (v1). This returns the database to the state it had before the transaction began.

Committed transactions are handled the opposite way: their commit record proves the work was intended to be durable, so their new values (v2) are redone going forward, in case the changes never reached disk. Undo before redo ensures the final state reflects only committed work.